

SambaNova Systems: What It Is and How It Integrates with PyTorch and vLLM

Background

SambaNova Systems is an AI infrastructure company that builds its own custom AI processor called the Reconfigurable Dataflow Unit (RDU). Unlike GPU vendors, SambaNova does not build on top of existing compute architectures — their chip is purpose-designed from scratch for AI inference workloads.

On Standard Hardware (NVIDIA, AMD, Intel)

Every mathematical operation your model runs — matrix multiply, softmax, attention — is backed by a kernel: a small, compiled program that runs directly on the hardware. NVIDIA writes these kernels in CUDA. AMD writes equivalent kernels in ROCm. Intel writes them in XPU.

PyTorch has an internal routing system called the dispatcher. When your model runs an operation, the dispatcher looks at which device the data is on and routes the call to the correct vendor's registered kernel. This routing happens live, in real time, as your code executes.

Hardware vendors plug into this system through a standard interface called PrivateUse1. Each vendor writes a kernel for every operation PyTorch supports, registers them under their device name, and their hardware becomes a native PyTorch device. You do not change your model code at all — you just move your data to the right device and PyTorch handles the rest.

`torch.matmul(a, b)`

- dispatcher checks which device the tensor is on
- "cuda" → runs NVIDIA's registered CUDA kernel
- "rocm" → runs AMD's registered ROCm kernel
- "xpu" → runs Intel's registered XPU kernel

PyTorch is the execution engine from start to finish. The vendor just supplies the kernels it calls into.

Why SambaNova Cannot Follow That Path

SambaNova's RDU is not a GPU. It does not run threads. It does not have a CUDA runtime or anything equivalent. Instead, it uses a dataflow architecture — and the difference from a GPU is fundamental.

A GPU operates instruction by instruction: it receives an operation, executes it, returns the result, and waits for the next instruction. Each operation is independent, scheduled at runtime. An RDU works differently. Before any data arrives, the entire model is compiled into a static graph that is wired directly onto the physical hardware units of the chip. The result of operation A flows directly into operation B through a programmable switching fabric that is configured for that specific model before execution begins. There is no runtime dispatch — the chip is configured for a specific model before execution begins.

This architectural difference means the GPU integration model does not apply. There is no way to write a "matmul kernel" for an RDU and register it with PyTorch's dispatcher, because the RDU does not execute individual operations — it executes a compiled, fused, pre-optimized representation of the entire model graph at once.

Because of this, SambaNova built their own execution stack called SambaFlow. They still accept PyTorch models as input — because every developer already writes in PyTorch — but what they do with that model is fundamentally different from what a GPU vendor does.

How SambaNova Actually Runs a PyTorch Model

When a developer wants to run a PyTorch model on an RDU, they go through three stages. None of these stages involve PyTorch executing anything.

Stage 1 — Convert the model

```
samba.from_torch_model(model)
```

This call mutates the model in place. Every `torch.Tensor` becomes a `SambaTensor`. Every `nn.Parameter` becomes a `SambaParameter`. Every layer's forward function — `nn.Linear`, `nn.Conv2d`, etc. — gets overridden with SambaFlow's own implementation. After this call, the model looks like a PyTorch model but is no longer one. You cannot run it with a regular CUDA tensor anymore.

Stage 2 — Compile the model

```
samba.session.compile(model, dummy_inputs)
```

SambaFlow traces through the model using fixed-shape dummy inputs and records every operation — what they are, in what order, and the exact shape of every tensor at every step. It then compiles this entire static graph into a PEF (Processor Executable Format) binary file. This binary maps each operation to specific physical hardware units on the RDU chip and encodes all the dataflow optimizations for that specific model and input shape.

This is called AOT (ahead-of-time) compilation. The entire model is compiled once, into a hardware-specific binary, before any real data is processed.

Stage 3 — Run on the RDU

```
samba.session.run(inputs)
```

The SambaFlow runtime loads the compiled PEF binary onto the RDU and executes it. PyTorch is not involved at this stage in any way. The execution engine is SambaFlow. The hardware being driven is the RDU.

SambaNova's vLLM Support — What It Actually Means

SambaNova markets vLLM support, and the claim is technically accurate — but the implementation matters significantly for this assessment.

There are two ways SambaNova interacts with vLLM today.

The first is that vLLM treats SambaNova's cloud service as a remote API endpoint. vLLM sends a request, SambaNova's cloud runs the model on RDU hardware internally, and returns the result. In this mode, vLLM has no interaction with the RDU hardware at all. It is no different from calling OpenAI's API — vLLM is just a client talking to a remote service.

The second is a split-hardware setup that SambaNova demonstrated publicly in mid-2026. To understand this, it helps to know that LLM inference has two distinct phases:

- Prefill — processing the entire input prompt at once. This is compute-heavy: many parallel matrix multiplications run simultaneously, and GPUs are well-suited to this.
- Decode — generating the response one token at a time. This phase is bottlenecked not by raw compute but by memory bandwidth — the model's weights must be read from memory for every single token generated. RDUs have a memory architecture optimized for this access pattern, which is the basis of SambaNova's claimed performance advantage during decode.

In this split-hardware setup, a GPU rack runs the prefill phase and a SambaNova RDU rack runs the decode phase. vLLM manages only the GPU side. The RDU side runs through SambaFlow. The two sides pass data to each other through a data transfer library. This is not yet generally available and requires owning both GPU and RDU hardware simultaneously.

What does not exist today is vLLM running natively and directly on RDU hardware. The most fundamental blocker is PagedAttention, the mechanism vLLM uses to manage GPU memory dynamically across concurrent requests. It has no equivalent on an RDU, because the RDU's memory layout is fixed at compile time inside the PEF binary and cannot be reconfigured at runtime the way PagedAttention requires. Beyond that, vLLM depends on FlashAttention, which requires custom CUDA kernels, and on continuous batching and prefix caching, which are built on top of PagedAttention's runtime memory management. None of these have an equivalent implementation in SambaFlow.

For vLLM to run natively on SambaNova hardware, SambaNova would need to either rebuild all of these features inside SambaFlow, or implement a PrivateUse1 backend that makes the RDU visible to PyTorch and vLLM as a standard device — which would require abandoning the AOT compilation model that is central to how the chip actually works. Neither path is trivial, and neither currently exists.